Chapter 4: Design & Implementation — Technical Inventory

4.1 Technology Stack Summary

Layer	Technology
Frontend	Next.js (React), TypeScript, Tailwind CSS
Backend	Node.js, Express, TypeScript
Database	PostgreSQL (via pg Pool)
Auth	JWT (httpOnly cookies + Bearer tokens), bcryptjs
File Uploads	Multer (disk storage)
Email	Resend API (dev fallback: console.log)
AI Chatbot	Google Gemini 2.5 Flash (@google/generative-ai)
Spreadsheet Export	xlsx (SheetJS)
Containerization	Docker (implied by rebuild-docker.sh and schema in

4.2 PostgreSQL Database Schema

4.2.1 Table: users

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
name	VARCHAR(160)	NOT NULL
email	VARCHAR(255)	UNIQUE, NOT NULL
password	VARCHAR(255)	NOT NULL
role	VARCHAR(50)	NOT NULL, DEFAULT 'member'
is_admin	BOOLEAN	DEFAULT FALSE
interests	TEXT	—
bio	TEXT	—
avatar_url	VARCHAR(500)	—
banner_image	VARCHAR(500)	—
email_verified	BOOLEAN	DEFAULT FALSE
verification_token	VARCHAR(255)	—
reset_token	VARCHAR(255)	—
resettokenexpires	TIMESTAMP WITH TIME ZONE	—
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT NOW()

Notes: Soft-delete via deleted_at column (referenced in queries but not in base schema DDL — added at application layer). The role field stores 'member', 'organizer', or 'admin'.

4.2.2 Table: communities

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
name	VARCHAR(200)	NOT NULL
description	TEXT	NOT NULL
category	VARCHAR(120)	—
website	VARCHAR(255)	—
owner_id	INTEGER	REFERENCES users(id) ON DELETE CASCADE
member_count	INTEGER	DEFAULT 0
banner_image	VARCHAR(500)	—
logo	VARCHAR(500)	—
facebook	VARCHAR(255)	—
instagram	VARCHAR(255)	—
linkedin	VARCHAR(255)	—
tiktok	VARCHAR(255)	—
location	VARCHAR(255)	—
is_verified	BOOLEAN	DEFAULT FALSE
is_private	BOOLEAN	DEFAULT FALSE
member_approval	BOOLEAN	DEFAULT FALSE
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT NOW()

4.2.3 Table: events

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
community_id	INTEGER	NOT NULL, REFERENCES communities(id) ON DELETE CAS
title	VARCHAR(200)	NOT NULL
description	TEXT	NOT NULL
event_date	TIMESTAMP WITH TIME ZONE	NOT NULL
location	VARCHAR(255)	—
attendee_count	INTEGER	DEFAULT 0
banner_image	VARCHAR(500)	—
start_time	VARCHAR(50)	—
end_date	TIMESTAMP WITH TIME ZONE	—
end_time	VARCHAR(50)	—
duration	VARCHAR(100)	—
event_type	VARCHAR(50)	DEFAULT 'physical'
max_attendees	INTEGER	—
allow_guests	BOOLEAN	DEFAULT FALSE
guest_limit	INTEGER	—
rsvp_deadline	TIMESTAMP WITH TIME ZONE	—
payment_type	VARCHAR(50)	DEFAULT 'free'
topics	TEXT	—
hosts	TEXT	—
speakers	TEXT	—
agenda	TEXT	—
requirements	TEXT	—
instructions	TEXT	—
age_limit	VARCHAR(100)	—
requires_documents	BOOLEAN	DEFAULT FALSE
document_instructions	TEXT	—
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT NOW()

4.2.4 Table: rsvps

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
user_id	INTEGER	NOT NULL, REFERENCES users(id) ON DELETE CASCADE
event_id	INTEGER	NOT NULL, REFERENCES events(id) ON DELETE CASCADE
status	VARCHAR(50)	NOT NULL, DEFAULT 'attending'
full_name	VARCHAR(255)	—
phone	VARCHAR(100)	—
email	VARCHAR(255)	—
answers	JSONB	—
document_url	VARCHAR(500)	—
document_name	VARCHAR(255)	—
document_status	VARCHAR(20)	—
documentreviewedat	TIMESTAMP WITH TIME ZONE	—
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT NOW()

Constraint: UNIQUE(userid, eventid) — one RSVP per user per event.

4.2.5 Table: event_questions

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
event_id	INTEGER	NOT NULL, REFERENCES events(id) ON DELETE CASCADE
question	TEXT	NOT NULL
type	VARCHAR(20)	NOT NULL, DEFAULT 'text'
required	BOOLEAN	NOT NULL, DEFAULT FALSE
options	JSONB	—
file_accept	VARCHAR(20)	DEFAULT 'both'
filemaxsize	INTEGER	DEFAULT 5
sort_order	INTEGER	NOT NULL, DEFAULT 0
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT NOW()

Supported type values: 'text', 'textarea', 'select', 'checkboxes', 'radio', 'date', 'number', 'file'.

4.2.6 Table: reviews

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
user_id	INTEGER	NOT NULL, REFERENCES users(id) ON DELETE CASCADE
community_id	INTEGER	REFERENCES communities(id) ON DELETE CASCADE
event_id	INTEGER	REFERENCES events(id) ON DELETE CASCADE
rating	INTEGER	NOT NULL, CHECK (rating >= 1 AND rating <= 5)
comment	TEXT	—
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT NOW()

4.2.7 Table: community_members

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
user_id	INTEGER	NOT NULL, REFERENCES users(id) ON DELETE CASCADE
community_id	INTEGER	NOT NULL, REFERENCES communities(id) ON DELETE CAS
joined_at	TIMESTAMP WITH TIME ZONE	DEFAULT NOW()

Constraint: UNIQUE(userid, communityid).

4.2.8 Table: organizer_applications

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
user_id	INTEGER	NOT NULL, REFERENCES users(id) ON DELETE CASCADE
community_name	VARCHAR(200)	NOT NULL
description	TEXT	NOT NULL
address	TEXT	—
contact_info	VARCHAR(255)	—
phone	VARCHAR(100)	—
social_media	TEXT	—
target_audience	VARCHAR(255)	—
age_group	VARCHAR(100)	—
category	VARCHAR(120)	—
website	VARCHAR(255)	—
documents_url	TEXT	—
motivation	TEXT	—
expected_members	VARCHAR(100)	—
meeting_frequency	VARCHAR(100)	—
experience	TEXT	—
venue_details	TEXT	—
org_type	VARCHAR(100)	—
year_established	DATE	—
facebook	VARCHAR(255)	—
instagram	VARCHAR(255)	—
linkedin	VARCHAR(255)	—
tiktok	VARCHAR(255)	—
contactpersonname	VARCHAR(255)	—
position_role	VARCHAR(255)	—
activities	TEXT	—
benefits	TEXT	—
info_accurate	BOOLEAN	DEFAULT FALSE
preferred_username	VARCHAR(160)	—
authorized_representative	BOOLEAN	DEFAULT FALSE
certificate_file	VARCHAR(255)	—
logo_file	VARCHAR(255)	—
additionaldocfile	VARCHAR(255)	—
status	VARCHAR(50)	NOT NULL, DEFAULT 'pending'
admin_notes	TEXT	—
temp_password	VARCHAR(255)	—
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT NOW()
reviewed_at	TIMESTAMP WITH TIME ZONE	—
reviewed_by	INTEGER	REFERENCES users(id) ON DELETE SET NULL

4.2.9 Table: announcements

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
community_id	INTEGER	NOT NULL, REFERENCES communities(id) ON DELETE CAS
title	VARCHAR(255)	NOT NULL
content	TEXT	NOT NULL
created_by	INTEGER	NOT NULL, REFERENCES users(id) ON DELETE CASCADE
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT NOW()
updated_at	TIMESTAMP WITH TIME ZONE	DEFAULT NOW()

4.2.10 Table: community_discussions

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
community_id	INTEGER	NOT NULL, REFERENCES communities(id) ON DELETE CAS
user_id	INTEGER	NOT NULL, REFERENCES users(id) ON DELETE CASCADE
content	TEXT	NOT NULL
parent_id	INTEGER	REFERENCES community_discussions(id) ON DELETE CAS
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT NOW()

Note: parentid enables threaded replies — top-level posts have parentid IS NULL.

4.2.11 Table: activity_log

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
user_id	INTEGER	REFERENCES users(id) ON DELETE SET NULL
user_name	VARCHAR(160)	—
action	VARCHAR(100)	NOT NULL
description	TEXT	—
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT NOW()

4.2.12 Table: notifications

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
user_id	INTEGER	NOT NULL, REFERENCES users(id) ON DELETE CASCADE
type	VARCHAR(100)	NOT NULL
title	VARCHAR(255)	NOT NULL
message	TEXT	—
link	VARCHAR(500)	—
is_read	BOOLEAN	DEFAULT FALSE
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT NOW()

Status: Table exists in schema but no API endpoints or backend logic currently create or query notifications. The notifications table is defined but not wired into the application.

4.2.13 Table: saved_events

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
user_id	INTEGER	NOT NULL, REFERENCES users(id) ON DELETE CASCADE
event_id	INTEGER	NOT NULL, REFERENCES events(id) ON DELETE CASCADE
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT NOW()

Constraint: UNIQUE(userid, eventid).

4.2.14 Table: community_sponsors

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
community_id	INTEGER	NOT NULL, REFERENCES communities(id) ON DELETE CAS
name	VARCHAR(255)	NOT NULL
website	VARCHAR(255)	—
logo	VARCHAR(500)	—
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT NOW()

Status: Table exists in schema; a GET endpoint (/api/communities/:id/sponsors) reads sponsors, but no POST/PUT/DELETE endpoint for creating or managing sponsors is implemented.

4.3 Entity-Relationship Summary

```
users (1) ——— (M) communities       [owner_id]
users (M) ——— (M) communities       [via community_members]
users (1) ——— (M) events            [via RSVPs]
communities (1) ——— (M) events      [community_id]
events (1) ——— (M) rsvps            [event_id]
events (1) ——— (M) event_questions  [event_id]
events (1) ——— (M) reviews          [event_id]
communities (1) ——— (M) reviews     [community_id]
communities (1) ——— (M) announcements [community_id]
communities (1) ——— (M) discussions  [community_id]
communities (1) ——— (M) sponsors    [community_id]
users (1) ——— (M) organizer_applications [user_id]
users (1) ——— (M) activity_log      [user_id]
users (1) ——— (M) saved_events      [user_id]
```

4.4 Backend API Endpoints (Complete Inventory)

4.4.1 Authentication — /api/auth (auth.ts)

Method	Path	Auth	Description
POST	/api/auth/register	None	Register new user (sends verification email)
POST	/api/auth/login	None	Login (returns JWT in httpOnly cookie)
POST	/api/auth/logout	None	Clear auth cookie
POST	/api/auth/forgot-password	None	Send password reset email
POST	/api/auth/reset-password	None	Reset password with token
POST	/api/auth/verify-email	None	Verify email with token
GET	/api/auth/me	Auth	Get current user profile
PUT	/api/auth/profile	Auth	Update name/bio/interests
PUT	/api/auth/change-password	Auth	Change password (temp password flow)
POST	/api/auth/avatar	Auth	Upload profile avatar (Multer, 2MB)
DELETE	/api/auth/avatar	Auth	Remove avatar
POST	/api/auth/banner	Auth	Upload profile banner (Multer, 5MB)
DELETE	/api/auth/banner	Auth	Remove banner

4.4.2 Communities — /api/communities (communities.ts)

Method	Path	Auth	Description
GET	/api/communities	None	List communities (search, category, sort, paginati
GET	/api/communities/categories	None	List distinct categories
GET	/api/communities/my-owned	Auth	Get communities owned by current user
GET	/api/communities/:id	None	Get community details
POST	/api/communities	Auth	Create community (Multer: banner + logo)
PUT	/api/communities/:id	Auth	Update community (owner or admin)
DELETE	/api/communities/:id	Auth	Soft-delete community (owner or admin)
GET	/api/communities/:id/members	Auth	Get member list (owner/admin only)
GET	/api/communities/:id/events	None	Get community events (upcoming/past)
GET	/api/communities/:id/membership	Auth	Check membership status
DELETE	/api/communities/:id/members/:userId	Auth	Remove a member (owner only)
POST	/api/communities/:id/join	Auth	Join community
POST	/api/communities/:id/leave	Auth	Leave community
GET	/api/communities/:id/sponsors	None	Get community sponsors
GET	/api/communities/:id/export	Auth	Download members as XLSX (owner/admin only)

4.4.3 Events — /api/events (events.ts)

Method	Path	Auth	Description
GET	/api/events	None	List events (filtering, search, pagination)
GET	/api/events/upcoming	None	Get upcoming events
GET	/api/events/my-saved	Auth	Get user's saved events
GET	/api/events/organizer	Auth	Get events for organizer's communities
GET	/api/events/:id	None	Get event details (with questions)
POST	/api/events	Auth	Create event (owner/admin, Multer banner)
PUT	/api/events/:id	Auth	Update event (owner/admin)
DELETE	/api/events/:id	Auth	Soft-delete event (owner/admin)
POST	/api/events/:id/save	Auth	Toggle save/unsave event
GET	/api/events/:id/saved	Auth	Check if event is saved
GET	/api/events/:id/attendees	None	Get event attendees
GET	/api/events/:id/export	Auth	Download attendee list as XLSX (2 sheets: attendee

4.4.4 Engagement — /api/engagement (engagement.ts)

Method	Path	Auth	Description
POST	/api/engagement/rsvp	Auth	RSVP to event (multipart: document + answer files)
DELETE	/api/engagement/rsvp	Auth	Cancel RSVP
GET	/api/engagement/rsvp/event/:eventId	Auth	Get RSVPs for event (organizer/admin)
PATCH	/api/engagement/rsvp/document-status	Auth	Review verification document
PATCH	/api/engagement/rsvp/answer-status	Auth	Review file answer to registration question
GET	/api/engagement/rsvp/user	Auth	Get current user's RSVPs
GET	/api/engagement/rsvp/status/:eventId	Auth	Check RSVP status for event
POST	/api/engagement/review	Auth	Create/update review (community or event)
GET	/api/engagement/review/community/:communityId	None	Get community reviews + avg rating
GET	/api/engagement/review/event/:eventId	None	Get event reviews + avg rating
GET	/api/engagement/community/my-joined	Auth	Get user's joined communities
GET	/api/engagement/community/is-member/:communityId	Auth	Check membership
POST	/api/engagement/community/join/:communityId	Auth	Join community
POST	/api/engagement/community/leave/:communityId	Auth	Leave community
GET	/api/engagement/community/members/:communityId	Auth	Get member list (owner/admin)
GET	/api/engagement/community/count/:communityId	None	Get member count
GET	/api/engagement/event/count/:eventId	None	Get attendee count

4.4.5 Admin — /api/admin (admin.ts)

Method	Path	Auth	Description
GET	/api/admin/stats	Admin	Dashboard statistics
GET	/api/admin/users	Admin	List all users (role filter, search)
GET	/api/admin/users/:userId	Admin	Get user details
PUT	/api/admin/users/:userId/role	Admin	Update user role
DELETE	/api/admin/users/:userId	Admin	Soft-delete user
GET	/api/admin/communities	Admin	List all communities
PUT	/api/admin/communities/:id	Admin	Edit any community
GET	/api/admin/communities/:id/details	Admin	Community details with demographics
DELETE	/api/admin/communities/:id	Admin	Soft-delete community
GET	/api/admin/events	Admin	List all events
PUT	/api/admin/events/:id	Admin	Edit any event
DELETE	/api/admin/events/:id	Admin	Soft-delete event
GET	/api/admin/trash	Admin	List all trashed items
POST	/api/admin/trash/:type/:id/restore	Admin	Restore trashed item
DELETE	/api/admin/trash/:type/:id/permanent	Admin	Permanently delete
POST	/api/admin/trash/cleanup	Admin	Purge items older than 30 days

4.4.6 Admin Activity — /api/admin (adminActivity.ts)

Method	Path	Auth	Description
GET	/api/admin/activity	Admin	Activity log (filterable, paginated)
GET	/api/admin/activity/stats	Admin	Activity summary (30-day counts by action)

4.4.7 Applications — /api/applications & /api/admin/applications (applications.ts)

Method	Path	Auth	Description
POST	/api/applications	Auth	Submit organizer application (Multer: cert, logo,
GET	/api/applications/my	Auth	Get current user's application
GET	/api/admin/applications	Admin	List all applications (status filter)
GET	/api/admin/applications/:id	Admin	Get application details
PUT	/api/admin/applications/:id/review	Admin	Approve or reject (generates temp password on appr

4.4.8 Announcements — /api/announcements (announcements.ts)

Method	Path	Auth	Description
GET	/api/announcements/:communityId	None	Get announcements for community
POST	/api/announcements/:communityId	Auth	Create announcement (owner/admin)
PUT	/api/announcements/:communityId/:announcementId	Auth	Edit announcement (owner/admin)
DELETE	/api/announcements/:communityId/:announcementId	Auth	Delete announcement (owner/admin)

4.4.9 Discussions — /api/discussions (discussions.ts)

Method	Path	Auth	Description
GET	/api/discussions/:communityId	None	Get discussions with replies (paginated)
POST	/api/discussions/:communityId	Auth	Post discussion or reply (members only)
DELETE	/api/discussions/:communityId/:discussionId	Auth	Delete (author, owner, or admin)

4.4.10 Recommendations — /api/recommendations (recommendations.ts)

Method	Path	Auth	Description
GET	/api/recommendations	Auth	Get community recommendations based on interests

4.4.11 Chat — /api/chat (chat.ts)

Method	Path	Auth	Description
POST	/api/chat	Optional	Send message to Gemini AI chatbot

4.4.12 Users — /api/users (users.ts)

Method	Path	Auth	Description
GET	/api/users	None	List users (public info only: id, name, avatar, ro
GET	/api/users/:id	None	Get public user profile + community memberships

4.4.13 Public Endpoints (index.ts)

Method	Path	Auth	Description
GET	/	None	Backend health check
GET	/api/stats	None	Platform stats (users, events, communities, connec
GET	/api/health	None	Database health check

4.5 Frontend Pages/Routes

Route	File	Description
/	app/page.tsx	Homepage (hero, upcoming events, featured communit
/login	app/login/page.tsx	Login page
/register	app/register/page.tsx	Registration page
/forgot-password	app/forgot-password/page.tsx	Forgot password form
/reset-password	app/reset-password/page.tsx	Reset password form (token-based)
/verify-email	app/verify-email/page.tsx	Email verification page
/onboarding	app/onboarding/page.tsx	Post-registration interest selection
/profile	app/profile/page.tsx	User profile (edit, avatar, banner)
/communities	app/communities/page.tsx	Community listing (search, filter, sort)
/communities/[id]	app/communities/[id]/page.tsx	Community detail page
/communities/[id]/edit	app/communities/[id]/edit/page.tsx	Edit community (owner)
/events	app/events/page.tsx	Event listing (search, filter, sort)
/events/[id]	app/events/[id]/page.tsx	Event detail + RSVP + registration form
/events/[id]/edit	app/events/[id]/edit/page.tsx	Edit event (owner)
/events/create	app/events/create/page.tsx	Create event form
/my-events	app/my-events/page.tsx	User's events (RSVPs + saved)
/my-communities	app/my-communities/page.tsx	User's joined communities
/apply	app/apply/page.tsx	Organizer application form
/organizer	app/organizer/page.tsx	Organizer dashboard
/organization	app/organization/page.tsx	Organization/management page
/admin	app/admin/page.tsx	Admin dashboard
/admin/users/[id]	app/admin/users/[id]/page.tsx	Admin user detail
/admin/communities/[id]	app/admin/communities/[id]/page.tsx	Admin community detail
/admin/events/[id]	app/admin/events/[id]/page.tsx	Admin event detail
/admin/applications/[id]	app/admin/applications/[id]/page.tsx	Admin application review

Reusable Components: Navbar, AdminNavbar, Footer, Chatbot, Announcements, Discussions, CommunityJourney, ConfirmModal, ImageCropper, Pagination, RichMessage, SharePopup, Skeleton, Toast.

4.6 User Roles and Permissions

Role	Capabilities
member (default)	Browse communities/events, join communities, RSVP
organizer	All member permissions + create/edit/delete own ev
admin	All organizer permissions + manage all users (role

Authorization mechanism: Middleware (authMiddleware, adminMiddleware, optionalAuth) validates JWT tokens from httpOnly cookies or Authorization headers. Per-route ownership checks compare req.userId against owner_id in the database.

4.7 Community Application & Admin Approval Workflow

Implemented flow:

1. User submits application (POST /api/applications) with: community name, description, category, contact info, social media, documents (certificate, logo, additional doc — Multer upload, 3MB limit). Only one pending application per user is allowed.
2. Admin reviews (GET /api/admin/applications) — can filter by status (pending, approved, rejected). Views full application details.
3. Admin approves or rejects (PUT /api/admin/applications/:id/review):

- On approval:
- User's role is changed to 'organizer'
- A new community is auto-created from the application data (with is_verified = TRUE)
- The applicant is auto-added as the first member
- A temporary password is generated and stored (hashed) in organizerapplications.temppassword
- An approval email with the temp password is sent via Resend
- Activity is logged
- On rejection:
- Status is updated, admin notes saved
- A rejection email is sent with optional admin notes
- Activity is logged

4. First login with temp password: The login endpoint detects if the password matches the stored temp password and returns needsPasswordChange: true. The frontend redirects to a password change form. PUT /api/auth/change-password validates the temp password, updates the password, and clears the temp_password field.

4.8 Event Creation and RSVP Workflow

Event Creation

1. Community owner or admin creates an event via POST /api/events (multipart form data with optional banner image upload, Multer 10MB limit).
2. Required fields: communityid, title, description, eventdate.
3. Optional fields: starttime, enddate, endtime, duration, location, eventtype (default 'physical'), maxattendees, allowguests, guestlimit, rsvpdeadline, paymenttype (default 'free'), topics, hosts, speakers, agenda, requirements, instructions, agelimit, requiresdocuments, documentinstructions.
4. Custom registration questions are saved via replaceEventQuestions() — all existing questions are deleted and re-inserted from the JSON payload.

RSVP Workflow

1. User submits RSVP via POST /api/engagement/rsvp (multipart form data).
2. Required validation:

- All required registration questions must be answered.
- If event.requires_documents is true, a verification document must be uploaded (PDF, DOC,

DOCX, JPG, PNG, GIF, WEBP — 10MB limit).

- For file-type questions: uploaded files are validated against the question's file_accept setting (images, documents, or both).

3. Seat capacity check: Atomic UPDATE claims a seat only if attendeecount < maxattendees (or unlimited if max_attendees IS NULL). This prevents oversubscription.
4. Upsert: ON CONFLICT (userid, eventid) DO UPDATE — a user can change their RSVP status.
5. Status transitions:

- Attending to Not Attending: decrements attendee_count.
- Not Attending to Attending: auto-joins the community, increments attendee_count.

6. Cancellation: DELETE /api/engagement/rsvp — removes RSVP, decrements count, deletes uploaded document from disk.
7. Document review: Organizer/admin can mark documents as verified or rejected via PATCH /api/engagement/rsvp/document-status. File-type registration answers can also be individually reviewed via PATCH /api/engagement/rsvp/answer-status.

4.9 Event Registration Questions and File Upload Functionality

Question Types (Implemented)

Type	Display	Input Method
text	Short answer	Text input
textarea	Long answer	Textarea
select	Dropdown	Select element
radio	Multiple choice	Radio buttons
checkboxes	Checkboxes	Checkbox group
date	Date picker	Date input
number	Number input	Number input
file	File upload	File input

File Upload Configuration

For file-type questions, the organizer configures:

- file_accept: 'images' (JPG, PNG), 'documents' (PDF, DOC, DOCX), or 'both' (default)
- filemaxsize: 2, 5, 10, 15, or 20 MB (default 5MB)

File Upload Storage

- Avatar uploads: backend/uploads/avatars/ (2MB limit, JPEG/PNG/GIF/WebP)
- Banner uploads: backend/uploads/banners/ (5MB limit, JPEG/PNG/GIF/WebP)
- Community images: backend/uploads/communities/ (10MB limit, JPEG/PNG/GIF/WebP)
- Event banners: backend/uploads/events/ (10MB limit, JPEG/PNG/GIF/WebP)
- Application documents: backend/uploads/ (3MB limit, PDF/DOC/DOCX/JPG/PNG/GIF/SVG)
- RSVP verification documents + file answers: backend/uploads/rsvp-documents/ (10MB limit, PDF/DOC/DOCX/JPG/PNG/GIF/WebP)

All uploads use multer with disk storage and unique filename suffixes (timestamp + random). Old

files are deleted from disk when replaced.

4.10 Attendee/Follower CSV Export Functionality

Export format: XLSX (not CSV), using the xlsx (SheetJS) library.

Community Member Export

- Endpoint: GET /api/communities/:id/export
- Authorization: Community owner or admin only
- Sheet: "Members"
- Columns: Name, Email, Role, Joined Date
- Filename: {communityname}members.xlsx

Event Attendee Export

- Endpoint: GET /api/events/:id/export
- Authorization: Community owner or admin only
- Sheet 1: "Attendees" — Name, Email, Phone, Status, RSVP Date, Verification Document URL, Document Status, + one column per registration question
- Sheet 2: "Community Members" — Name, Email, Role, Joined Date
- Filename: {eventtitle}attendees.xlsx

4.11 Recommendation System Implementation

Endpoint: GET /api/recommendations (auth required)
Algorithm: Keyword-based content matching (not collaborative filtering or ML-based).
Data used:

- users.interests — comma-separated string of interest keywords set during onboarding
- communities — name, description, category, membercount, isverified
- community_members — to identify already-joined communities

Scoring logic:

1. User interests are split by comma and lowercased.
2. For each community, a searchText is built from name + description + category.
3. For each interest keyword:

- Regex word-boundary match is tested against the community's name (+5 points), category (+3 points), or description (+1 point).
- Substring match adds +0.5 points.

4. Popularity boost: member_count * 0.02.
5. Joined communities: score multiplied by 0.1 (effectively deprioritized).
6. Verified communities: +0.5 bonus.
7. Results sorted by score descending, then by member_count.
8. Returns top N (default 10, max 50). Option to exclude already-joined communities.

Not implemented: Collaborative filtering, event recommendations (only communities are recommended), machine learning models, or user behavior tracking.

4.12 Gemini API Chatbot Implementation

Endpoint: POST /api/chat (optional auth — guests can chat, logged-in users get personalized responses)
Model: Google Gemini 2.5 Flash via @google/generative-ai SDK.

Architecture: Two-Pass Flow

Pass 1 — SQL Generation:

1. The system builds a context header containing the current user's ID, name, and interests, plus the full database schema.
2. Template matching first: Common intents (upcoming events, this week/month, communities, recommendations) are matched via regex and executed with predefined SQL — no Gemini call needed (1 API call total).
3. If no template matches, Gemini is asked to generate a read-only SELECT query wrapped in <sql> tags.
4. SQL safety validation: isReadOnlyQuery() rejects non-SELECT queries; sanitizeSql() truncates at semicolons.

Pass 2 — Final Answer:

1. Real query results (formatted as pipe tables) are sent back to Gemini with the original question and formatting rules.
2. Gemini generates a user-facing markdown reply with <action> tags for clickable buttons.
3. Action tags are parsed, stripped from the response, and returned as structured action objects.

Deterministic Fallbacks

- If Gemini fails or returns errors (rate limit, safety, API key issues), buildFallbackReply() generates a clean markdown response from the query results without Gemini.
- Auto-generated action buttons are appended from real result rows (up to 3 per entity type).
- First-message greeting (Hello {Name}!) is enforced deterministically.

Template Intents (Predefined SQL)

Intent Pattern	SQL Template
"all events", "list events", "table"	Upcoming events, LIMIT 10
"this month" + month names	Events in current month, LIMIT 3
"this week", "this weekend"	Events in next 7 days, LIMIT 3
"upcoming", "what's happening"	Upcoming events, LIMIT 3
"recommend events"	Events ordered by interest-matched category
"all communities"	Communities by member count, LIMIT 10
"communities", "clubs", "groups"	Communities ordered by interest-matched category,

Configuration (chat.ts)

- SCHEMA: Full database schema string sent to Gemini for context.
- SYSTEM_PROMPT: Detailed instructions for response formatting, SQL rules, action button syntax, and greeting behavior.

4.13 Features Defined in Schema but NOT Implemented

Feature	Status
notifications table	Defined in schema, no API endpoints or backend log
community_sponsors CRUD	GET endpoint exists, but no POST/PUT/DELETE for ma
isprivate / memberapproval fields on communities	Stored in schema and editable, but no enforcement
allowguests / guestlimit on events	Stored and editable, but no enforcement logic in R
rsvp_deadline on events	Stored and editable, but no enforcement (RSVPs are
payment_type on events	Stored, but no payment integration exists
hosts, speakers, agenda, requirements, instructions	Stored and editable, but displayed as raw text — n
duration on events	Stored, but not validated or enforced
gender field on users	Referenced in admin community details query (gende
deleted_at columns	Referenced in all queries (WHERE deleted_at IS NUL
email_verified enforcement	Email verification is sent, but login does not che